Resolução do Problema das n-Rainhas por meio de Algoritmos de Busca com Informação

José Neres Gomes da Silva Filho, Tiago Moura e Silva

Tecnólogo em Análise e Desenvolvimento de Sistemas – Módulo III

Instituto Federal do Piauí (IFPI) – Campus Picos
Picos – PI – Brasil

Resumo. Este trabalho resolve o problema das n-rainhas com dois algoritmos de busca com informação, A* e Busca Gulosa, usando a formulação incremental e, como heurística, a quantidade de pares de rainhas que se atacam. Os algoritmos foram implementados em Python e executados para 8, 80 e 800 rainhas, medindo a quantidade de passos (nós expandidos) e o tempo. O A* resolveu apenas o caso de 8 rainhas, com 2.741.210 passos, pois seu esforço cresce de 9 a 14 vezes a cada rainha adicional. A Busca Gulosa resolveu os três tamanhos, em geral com um passo por rainha, mas depende do critério de desempate entre casas equivalentes.

1. Introdução

O problema das n-rainhas consiste em colocar n rainhas em um tabuleiro n × n sem que duas delas fiquem na mesma linha, na mesma coluna ou na mesma diagonal. Neste trabalho o problema foi resolvido com dois algoritmos de busca com informação: o A* e a Busca Gulosa.

Foi construído um programa em Python (arquivo rainhas.py) que recebe como entrada a quantidade de rainhas e o algoritmo desejado e mostra a solução encontrada, a quantidade de passos e o tempo gasto. Este relatório apresenta a formulação do problema e a análise da execução de cada algoritmo para 8, 80 e 800 rainhas, considerando a quantidade de passos e o tempo necessários para encontrar a solução.

2. Formulação do problema

A formulação usada é a incremental, apresentada por [Russell e Norvig 2013] para o problema das oito rainhas e aqui aplicada a n rainhas: o tabuleiro começa vazio e as rainhas são colocadas uma de cada vez. Foram feitas apenas as distinções necessárias para garantir uma solução válida. Importa saber em quais casas existem rainhas. Não importa a ordem em que foram colocadas, e as rainhas são todas iguais. Por isso um estado é um conjunto de casas, e o mesmo conjunto obtido por ordens diferentes é um único estado. Também não é permitido pôr duas rainhas na mesma casa nem passar de n rainhas, pois uma solução tem exatamente n.

Tabela 1. Formulação do problema

Elemento	Definição
Estado	Conjunto de casas do tabuleiro n × n que contêm rainha (de 0 a n rainhas, uma por casa).
Estado inicial	Tabuleiro vazio, com 0 rainhas.
Ação	Adicionar uma rainha em qualquer casa vazia. Um estado com k rainhas tem $n^2 - k$ sucessores.
Custo	Cada novo estado gerado custa 1. Assim, g é a quantidade de rainhas no tabuleiro.
Heurística h	Quantidade de pares de rainhas que se atacam, direta ou indiretamente (mesma linha, coluna ou diagonal, mesmo que haja outra rainha entre elas).
Teste de meta	O estado tem n rainhas e h = 0.

Para calcular h, conta-se quantas rainhas há em cada linha, coluna e diagonal. Uma linha com c rainhas contribui com $c(c - 1)/2$ pares, e o mesmo vale para colunas e diagonais. Ao colocar uma nova rainha, h aumenta pela soma das rainhas que já estão na linha, na coluna e nas duas diagonais daquela casa. Esse valor é calculado para todas as casas de uma vez, o que torna o programa rápido mesmo para 800 rainhas.

Colocar uma rainha nunca desfaz um ataque, então h nunca diminui ao longo de um caminho. Um estado com h > 0 jamais chega a uma meta, de modo que h é admissível. Como cada passo custa 1 e h não diminui, h também é consistente, e por isso o A* é ótimo e não precisa reabrir nós. Todo caminho até a meta custa exatamente n.

3. Algoritmos implementados

3.1. A* e Busca Gulosa

Os dois algoritmos usam a mesma estrutura: uma fronteira (fila de prioridade) com os estados ainda não expandidos e uma lista de estados já gerados, para não repetir estados. A única diferença é a prioridade. O A* escolhe o estado de menor f = g + h. A Busca Gulosa escolhe o de menor f = h. Quando o valor de f é igual, é expandido primeiro o estado com mais rainhas. O teste de meta é feito quando o estado sai da fronteira.

Gerar todos os sucessores de cada estado exigiria guardar, para 800 rainhas, 640 mil estados por passo. Por isso o programa gera os sucessores de um estado um por vez, do melhor para o pior. A ordem em que os estados são expandidos é a mesma, mas a memória gasta é muito menor. Toda solução encontrada é conferida no final, recalculando h do zero.

3.2. Desempate entre casas e tentativas da Busca Gulosa

Na formulação pedida, muitas casas têm o mesmo valor de h (todas as casas livres de ataque, por exemplo). A Busca Gulosa depende muito de como esses empates são resolvidos. Foram testadas duas formas. Na primeira, o empate é resolvido pela ordem das casas, linha a linha. Na segunda, chamada aqui de MRV, entre as casas com o mesmo

h o programa prefere aquela cuja linha tem menos casas livres de ataque, depois a de coluna com menos casas livres, e por último usa uma ordem sorteada a partir de uma semente. O desempate só atua quando os valores de h são iguais, então a heurística continua sendo a do enunciado.

Como o resultado depende do sorteio, o programa tenta a semente 0 e, se a tentativa passar do limite de passos (1,25 × n + 100), tenta a semente 1, e assim por diante até encontrar a solução. Os passos e o tempo da Busca Gulosa informados neste relatório somam todas as tentativas. O A* usa o desempate linha a linha e não depende de sorteio.

3.3. Execução do programa

O programa precisa do Python 3 e da biblioteca NumPy. Na pasta do trabalho, o comando python rainhas.py pergunta a quantidade de rainhas e o algoritmo (1 para A*, 2 para Busca Gulosa). Também é possível informar os dois valores de uma vez, por exemplo python rainhas.py 8 2. Até 20 rainhas o tabuleiro é desenhado na tela; para mais que isso, as posições são salvas em um arquivo de texto. O A* só termina para tabuleiros pequenos: com mais de 8 rainhas o programa avisa e desiste depois de 2 minutos. Os arquivos experimentos.py e graficos.py repetem os testes e os gráficos deste relatório.

4. Análise da execução

4.1. Como os testes foram feitos

Cada algoritmo foi executado para 8, 80 e 800 rainhas. Um passo é um nó expandido, isto é, um estado retirado da fronteira para gerar seus sucessores. O tempo é o tempo de relógio gasto pela busca. Os testes foram feitos em Python 3.12, com um único núcleo do processador e um teste por vez, para que um não atrapalhasse o outro. Os tempos mudam de um computador para outro, mas a quantidade de passos não muda.

O A* com 8 rainhas foi executado até o fim. Para 80 e 800 rainhas ele foi interrompido depois de 60 segundos, porque o número de estados a expandir era grande demais. A Busca Gulosa foi executada pelo procedimento da seção 3.2. Além disso, para cada tamanho foram feitas 20 tentativas independentes (sementes 0 a 19) para medir com que frequência uma tentativa dá certo.

4.2. Resultados com 8, 80 e 800 rainhas

Tabela 2. Passos e tempo de cada algoritmo

Algoritmo	Rainhas	Resolveu?	Passos	Tempo (s)	Tentativas
A*	8	Sim	2.741.210	375,8	1
A*	80	Não (parou em 60 s)	412.400*	60,0	1
A*	800	Não (parou em 60 s)	8.800*	60,4	1
Busca Gulosa	8	Sim	9	< 0,01	1
Busca Gulosa	80	Sim	81	0,01	1
Busca Gulosa	800	Sim	1.901	18,1	2

* Quantidade atingida até o programa ser interrompido.

A Tabela 2 e as Figuras 1 e 2 resumem os resultados. Com 8 rainhas o A* encontrou a solução com 2.741.210 passos em cerca de 375,8 segundos, usando aproximadamente 827 MB de memória. A Busca Gulosa resolveu o mesmo caso com 9 passos, que é o mínimo possível (uma expansão por rainha mais o estado inicial), em tempo menor que 0,01 segundo.

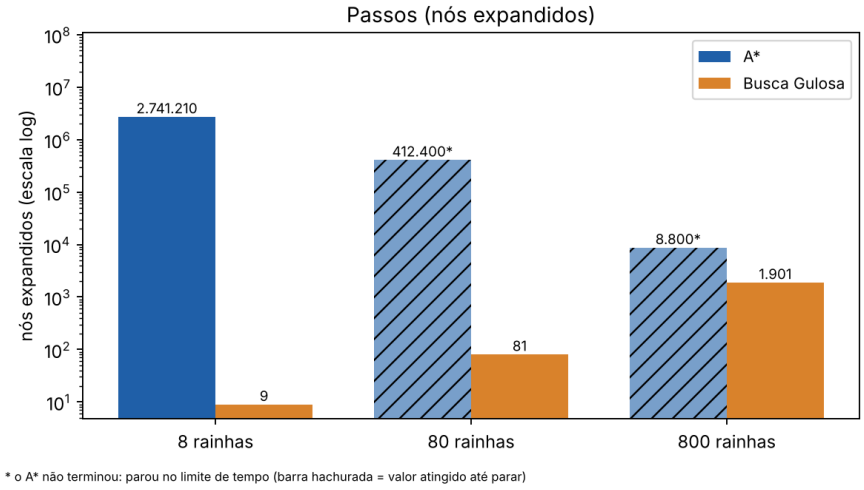


Figura 1. Passos (nós expandidos) por algoritmo

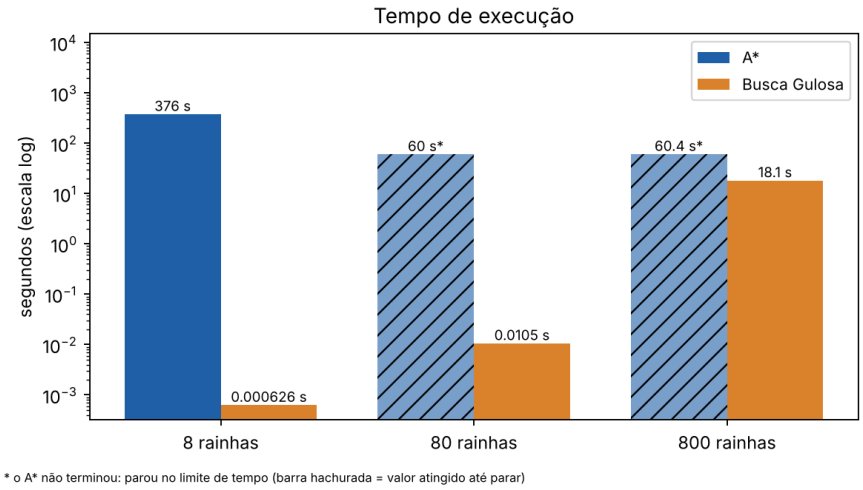


Figura 2. Tempo de execução por algoritmo

Com 80 rainhas a Busca Gulosa usou 81 passos (80 rainhas mais o estado inicial) e terminou em cerca de 0,01 segundo. O A* não terminou: em 60 segundos expandiu 412.400 estados e ainda estava no nível f = 2, muito longe da meta, que só é alcançada em f = 80.

Com 800 rainhas a Busca Gulosa precisou de 2 tentativas, somando 1.901 passos e 18,1 segundos. A tentativa que deu certo fez 801 passos, ou seja, também sem nenhum retrocesso. O tempo é alto em relação ao número de passos porque, a cada passo, o programa calcula o h de todas as 640.000 casas do tabuleiro. O A* expandiu 8.800 estados em 60 segundos e ficou em f = 1. A Figura 3 mostra uma das soluções obtidas para 8 rainhas.

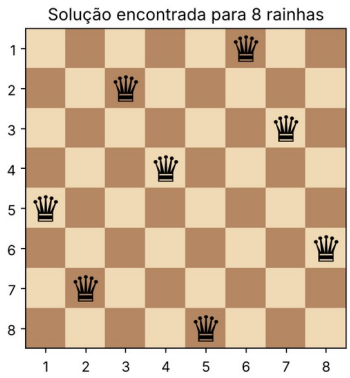


Figura 3. Solução encontrada para 8 rainhas

4.3. Crescimento do esforço do A*

Para entender por que o A* não terminou com 80 e 800 rainhas, ele foi executado também para 4, 5, 6 e 7 rainhas. A Tabela 3 e a Figura 4 mostram o resultado.

Tabela 3. Esforço do A* para n de 4 a 8

Rainhas	Passos	Estados gerados	Tempo (s)
4	162	2.248	0,02
5	1.445	32.039	0,16
6	15.118	491.336	1,42
7	192.983	8.647.691	22,1
8	2.741.210	161.978.332	375,8

Estados gerados: soma dos sucessores de todos os nós expandidos.

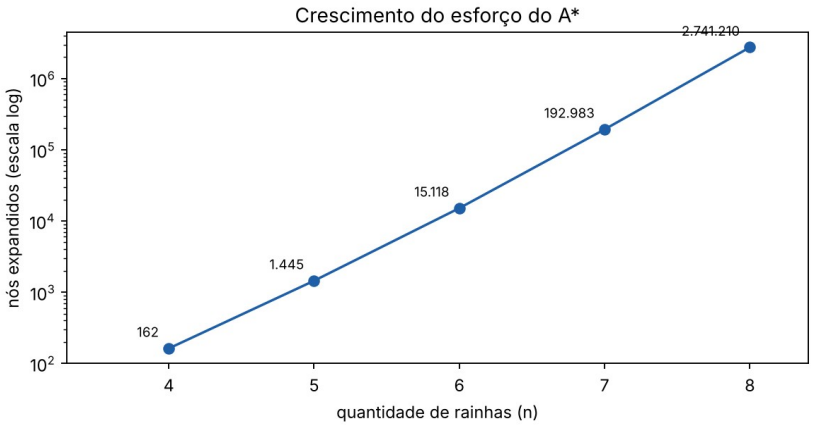


Figura 4. Crescimento do número de passos do A*

A cada rainha a mais, o número de passos é multiplicado por um valor entre 9 e 14. O motivo é o custo g. Como todas as soluções custam n e h é admissível, o A* só encontra a meta depois de expandir todos os estados com f menor que n. Isso inclui todos os arranjos sem ataques com menos de n rainhas e muitos arranjos com ataques, e a quantidade deles cresce de forma exponencial. Por isso o A* só é viável para tabuleiros pequenos. Além disso, como todas as soluções têm o mesmo custo n, a otimalidade do A* não traz vantagem neste problema.

4.4. Efeito do desempate na Busca Gulosa

A Tabela 4 compara os dois desempates da Busca Gulosa. Com o desempate linha a linha, ela resolveu 8 rainhas, mas não terminou 80 nem 800 rainhas em 30 segundos. Quando o primeiro caminho chega a um beco sem saída (todas as casas livres sob ataque), voltar e testar outras casas é muito caro, porque o mesmo conjunto de rainhas pode ser montado em ordens diferentes. Com o desempate MRV, a maior parte das tentativas bem-sucedidas não precisou voltar nenhuma vez.

Tabela 4. Busca Gulosa com cada tipo de desempate

Desempate	Rainhas	Resultado	Tentativas que deram certo
Linha a linha	8	Resolveu (1.970 passos)	–
Linha a linha	80	Não resolveu em 30 s (227.400* passos)	–
Linha a linha	800	Não resolveu em 30 s (4.200* passos)	–
MRV com sorteio	8	Resolveu	13 de 20
MRV com sorteio	80	Resolveu	11 de 20
MRV com sorteio	800	Resolveu	4 de 20

* Quantidade atingida até o programa ser interrompido.

Mesmo com o MRV, nem toda tentativa dá certo: foram 13 de 20 com 8 rainhas, 11 de 20 com 80 e apenas 4 de 20 com 800. Por isso o programa recomeça com outra semente quando uma tentativa passa do limite. Isso mostra uma característica da Busca Gulosa: ela é muito mais rápida que o A*, mas depende do desempate e não garante resolver na primeira tentativa.

5. Conclusão

Os dois algoritmos foram implementados conforme o enunciado, com a formulação incremental, a heurística de pares de rainhas que se atacam e custo 1 por estado gerado. O A* resolveu apenas o caso de 8 rainhas (2.741.210 passos e cerca de 375,8 segundos). Para 80 e 800 rainhas ele não terminou, pois o número de passos cresce muito rápido com n. A Busca Gulosa resolveu os três tamanhos, em geral com uma expansão por rainha, e levou cerca de 18,1 segundos para 800 rainhas. Em compensação, o seu sucesso depende do desempate escolhido e, com 800 rainhas, só uma parte das tentativas termina dentro do limite. Concluímos que, para este problema, a Busca Gulosa é a opção prática, enquanto o A* só serve para tabuleiros pequenos.

Referências

Russell, S. e Norvig, P. (2013). Inteligência Artificial. Elsevier, Rio de Janeiro, 3ª edição.